

# Bataille navale

Étude de la confiance accordée à un humain vs une IA

Mallory BARATAY - Patrice KEGLO  
Groupe CTRL-Z

Master 1 Génie Logiciel  
Faculté des Sciences de Montpellier  
Département Informatique

29 mai 2026

# Plan

- 1 Introduction
- 2 Contexte du Projet
- 3 Développement
- 4 Protocole Expérimental
- 5 Résultats Obtenus
- 6 Conclusion

# Introduction

- **Question fondamentale** : Les utilisateurs accordent-ils différemment leur confiance à un adversaire humain vs une IA ?
- **Contexte** : Omniprésence de l'IA dans les interactions quotidiennes
- **Enjeu** : Conception d'interfaces homme-machine éthiques et fiables

## Théorie de l'attribution

- Heider (1958), Weiner (1985)
- Explications causales biaisées pour les machines

## Étude Ullman et al. (2014)

- Yale University
- Confiance vs tromperie du robot

## Anthropomorphisme

- Standards moraux différents selon la cible
- Machines jugées plus sévèrement

# Hypothèses de recherche

- H<sub>1</sub>** Les participants expriment une évaluation de déloyauté **significativement plus élevée** pour le CheatBot que pour le même comportement attribué à un humain
- H<sub>2a</sub>** L'ordre de présentation de l'adversaire **influence** les perceptions de triche
- H<sub>2b</sub>** La confiance **diminue** au fil des tours

# Contexte du Projet

## Phase 1 : Robot Pepper

- Objectif initial : humain vs humanoïde vs digital
- Intégration avec robot social SoftBank

## Problèmes rencontrés

- Parties trop longues (15-30s latence)
- Confonds expérimentaux
- Expérience frustrante

---

## Phase 2 : Pivot vers digital

- Maintien de Python (infrastructure Pepper déjà développée)
- Ajout de couche web (Flask)
- Avantage : contrôle rigoureux, reproductibilité, scalabilité

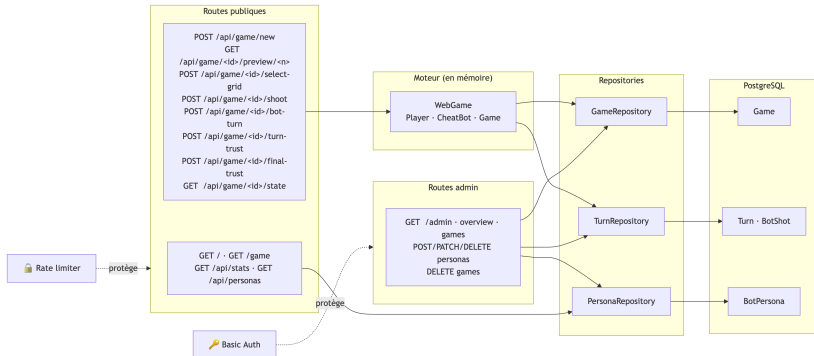


# Développement

## Composants clés :

- **Backend Python** : Logique métier, CheatBot, gestion de base de données
- **Frontend Web** : Interface utilisateur (Flask + JavaScript)
- **Communication** : API REST

# Plateforme Web : Vue d'ensemble



## Routes principales :

- GET / : Landing page
- POST /api/game/new : Créer partie
- POST /api/game/{id}/shoot : Tir joueur
- POST /api/game/{id}/bot-turn : Tour du bot
- POST /api/game/{id}/turn-trust : Enregistrer score
- GET /api/stats : Statistiques globales

 **PEPPER BATTLESHIP**

FONCTIONNALITÉSCOMMENT JOUERRECHERCHE

**x Jouer**

EXPÉRIENCE DE RECHERCHE · UNIVERSITÉ

# POUVEZ-VOUS DÉTECTER LA TRICHE ?

Affrontez notre robot Pepper dans une partie de Bataille Navale. Il maîtrise l'art de tricher discrètement. Saurez-vous le démasquer ?

30  
PARTIES JOUÉES

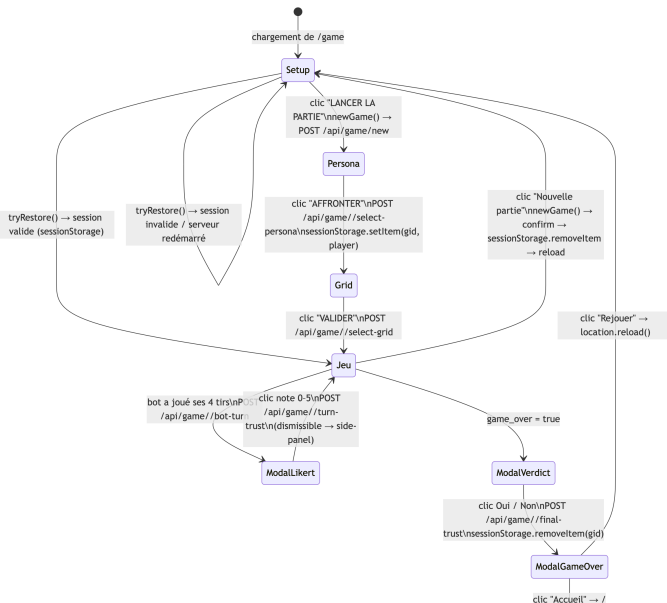
4  
TIRS PAR TOUR

10  
CONFIGURATIONS

6  
BATEAUX EN JEU

**x COMMENCER LA PARTIE**

# Machine d'état



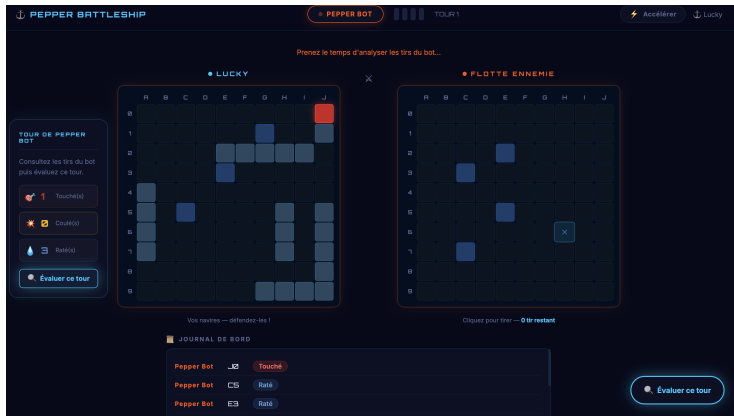
# Grilles de jeu

## Grille du joueur :

- Ma flotte (6 bateaux)
- Tirs ennemis reçus
- Couleurs

## Grille de suivi :

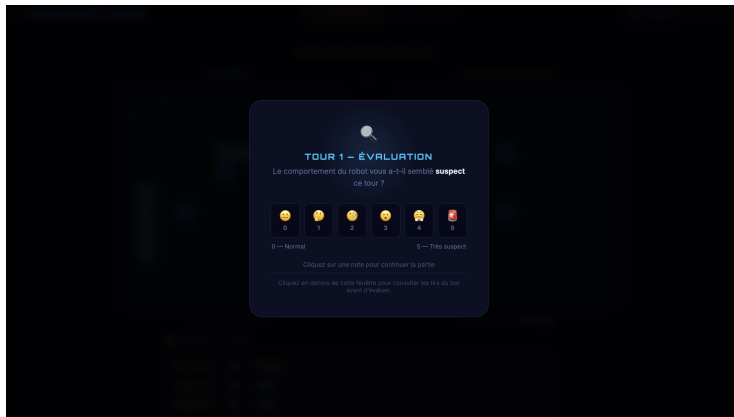
- Mes tirs effectués
- Résultats (touché/raté/coulé)
- Visualisation en temps réel



# Système de notation de confiance

Après chaque tour du bot :

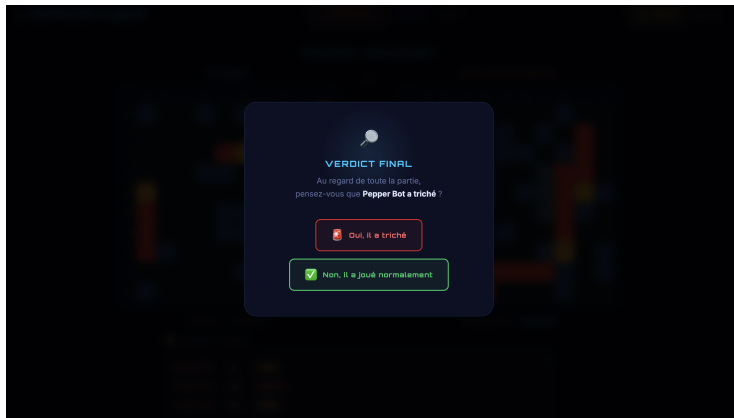
*“Le comportement de votre adversaire vous a-t-il semblé suspect ce tour ? ”*





A la fin de la partie :

*“Au regard de toute la partie, pensez-vous que votre adversaire a triché ?”*



## Algorithme de triche :

- Accès à la grille ennemie
- Quota prédéfini par tour : [1, 2, ..., 3]

## Chaque tour :

- 1 Identification d'un bateau ayant reçu au moins un tir
- 2 Sélection de cases à touché sur ce bateau (succès)
- 3 Sélection des cases vides (échecs)
- 4 Sélection d'un autre bateau pour compléter les tirs si besoin
- 5 Exécution des 4 tirs

# Base de données PostgreSQL

## Schéma de la base :

| Turn |             |    |
|------|-------------|----|
| int  | id          | PK |
| int  | game_id     | FK |
| int  | turn_number |    |
| int  | bot_quota   |    |
| int  | trust_score |    |

contient

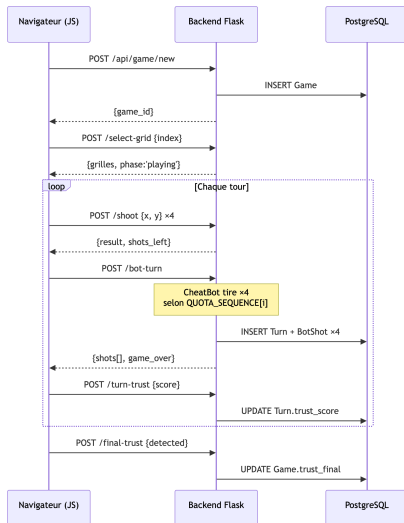
| BotShot |             |    |
|---------|-------------|----|
| int     | id          | PK |
| int     | turn_id     | FK |
| int     | shot_number |    |
| int     | pos_x       |    |
| int     | pos_y       |    |
| text    | result      |    |

| Game      |             |    |
|-----------|-------------|----|
| int       | id          | PK |
| text      | player_name |    |
| text      | player_type |    |
| timestamp | date_played |    |
| text      | winner      |    |
| int       | trust_score |    |
| int       | trust_final |    |

utilise

| BotPersona |             |    |
|------------|-------------|----|
| int        | id          | PK |
| text       | name        |    |
| text       | emoji       |    |
| text       | description |    |
| boolean    | active      |    |

## Flux de données complet :



# Protocole Expérimental

Chaque participant joue **2 parties** avec comportement adversaire **identique**

|             | <b>Partie 1</b> | <b>Partie 2</b> |
|-------------|-----------------|-----------------|
| Condition A | Humain          | IA              |
| Condition B | IA              | Humain          |

## Phases d'une partie :

- ① Sélection du nom du joueur
- ② Choix de l'adversaire
- ③ Sélection de la configuration de la flotte
- ④ Combat
- ⑤ Évaluation de perception de triche globale

## Variable dépendante principale :

À la fin de chaque tour du bot, le joueur répond :

*“Le comportement de votre adversaire vous a-t-il semblé suspect ce tour ? ”*

Échelle de Likert 0-5 :

- 0 = Pas du tout triché
- 1-2 = Peu probable
- 3 = Incertain
- 4-5 = Très suspect / Certainement triché

**Enregistrement** : Horodaté et stocké immédiatement en base



## Garantie d'équivalence comportementale :

- **Même adversaire** : Algorithme identique
- **Même quota** : Séquence [1, 2, 0, 3, 3, 2, 1, 4, 2, 1, 3] répétée

## Variable indépendante manipulée :

- Label de l'adversaire
- Ordre de présentation

## Variation entre joueurs :

- Chance du joueur à toucher le bot
- **Connaissance du jeu** pour le joueur

# Résultats Obtenus

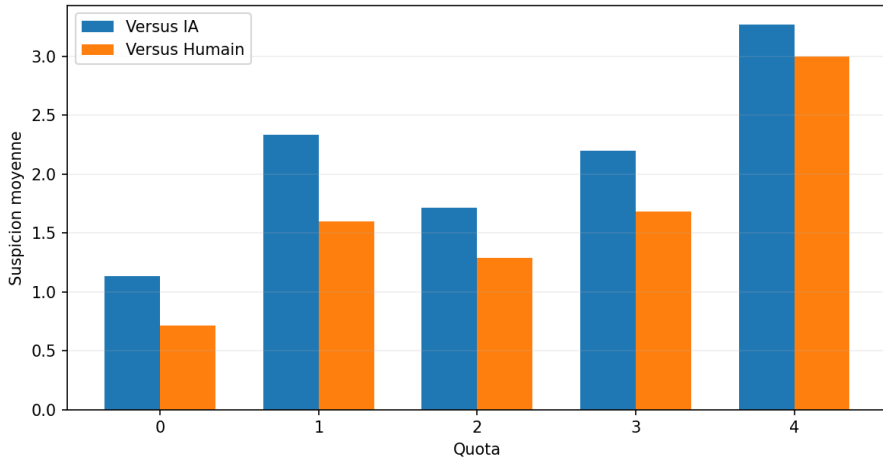
## Population :

- Taille de l'échantillon : 28 participants pour 40 parties
- Amis, collègues de l'université et famille
- Diversité d'âge, de genre et de familiarité avec les jeux vidéo
- Majorité masculine (environ 80%)

- $H_1$  : Différence significative entre les deux labels
- $H_{2a}$  : Effet d'ordre de présentation
- $H_{2b}$  : Diminution de la confiance au fil des tours

# Résultats obtenus $H_1$

Suspicion moyenne par quota

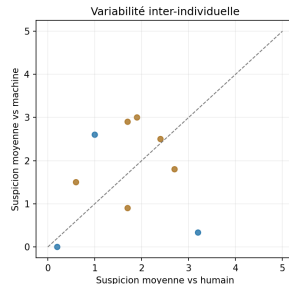
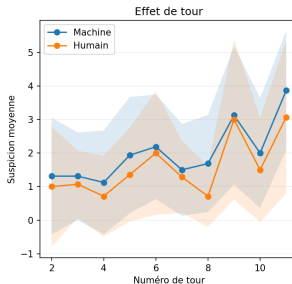
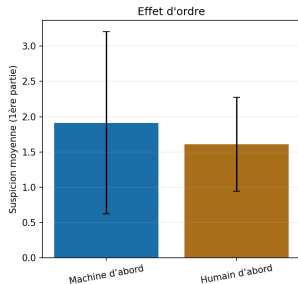


**$H_1$  est confirmée :**

- Les utilisateurs appliquent des standards différents selon le label
- Biais systématique dans la perception de tromperie artificielle

**Implications pour le design d'IA**

# Résultats obtenus $H_{2a}$ et $H_{2b}$



**$H_{2a}$  et  $H_{2b}$  sont confirmées :**

- Léger effet d'ordre
- Augmentation progressive de la suspicion au fil des tours
- Manque de donnée pour confirmer les hypothèses secondaires



## Interprétation :

- Biais potentiels à cause de notre proximité avec les joueurs
- Biais systématique dans la perception de tromperie artificielle
- Besoin de transparence et d'explicabilité pour les systèmes d'IA

# Conclusion

## Apports scientifiques :

- Éclairage sur la confiance homme-machine
- Protocole reproductible

## Perspectives futures :

- Augmentation des échantillons pour confirmer l'étude
- Réintégration du robot Pepper
- Étude des facteurs individuels modulant le biais
- Différents contextes compétitifs (poker, jeux de données)

- Architecture web+Python flexible et scalable
- Algorithme de triche contrôlé
- Collecte de données centralisée
- Intégrabilité avec robot physique disponible

## Bataille navale

*Détection de triche et confiance homme-machine*

Faculté des Sciences de Montpellier - Master 1 Génie Logiciel

Questions ?